Configuração do Backend.

Nesta etapa do projeto foi realizada a configuração inicial do ambiente de desenvolvimento do backend do sistema de verificação de preços, utilizando o framework Django.

Inicialmente, foi criado um ambiente virtual Python com o objetivo de isolar as dependências do projeto, para garantir organização e evitar conflitos com outras aplicações instaladas no sistema operacional.

Após a ativação do ambiente virtual, foram instaladas as principais dependências do projeto por meio do gerenciador de pacotes “pip”. As bibliotecas instaladas foram:

- **Django**: framework web responsável pela estrutura do backend;
- **Django REST Framework**: utilizado para a construção de APIs RESTful;
- **mysql-connector-python**: driver para comunicação entre o Django e o banco de dados MySQL.

Essas dependências foram escolhidas por serem amplamente utilizadas no mercado, estáveis e compatíveis com o sistema operacional Windows.

Em seguida, foi criado o projeto Django utilizando o comando:

“django-admin startproject core .”

Esse comando gerou a estrutura base do projeto, incluindo os arquivos de configuração principais, como: **“settings.py”**, **“urls.py”** e **“wsgi.py”**.

Posteriormente, foi criado um aplicativo Django denominado **produtos**, responsável por concentrar a lógica relacionada aos produtos e à consulta de preços:

“python manage.py startapp produtos”

O **core** é o projeto principal do Django. Ele centraliza as configurações do sistema, o roteamento principal das requisições e a integração entre os aplicativos.

Neste projeto, o app **produtos** é carregado pelo core e a partir das configurações feitas, acessa o banco de dados e disponibiliza a API REST.

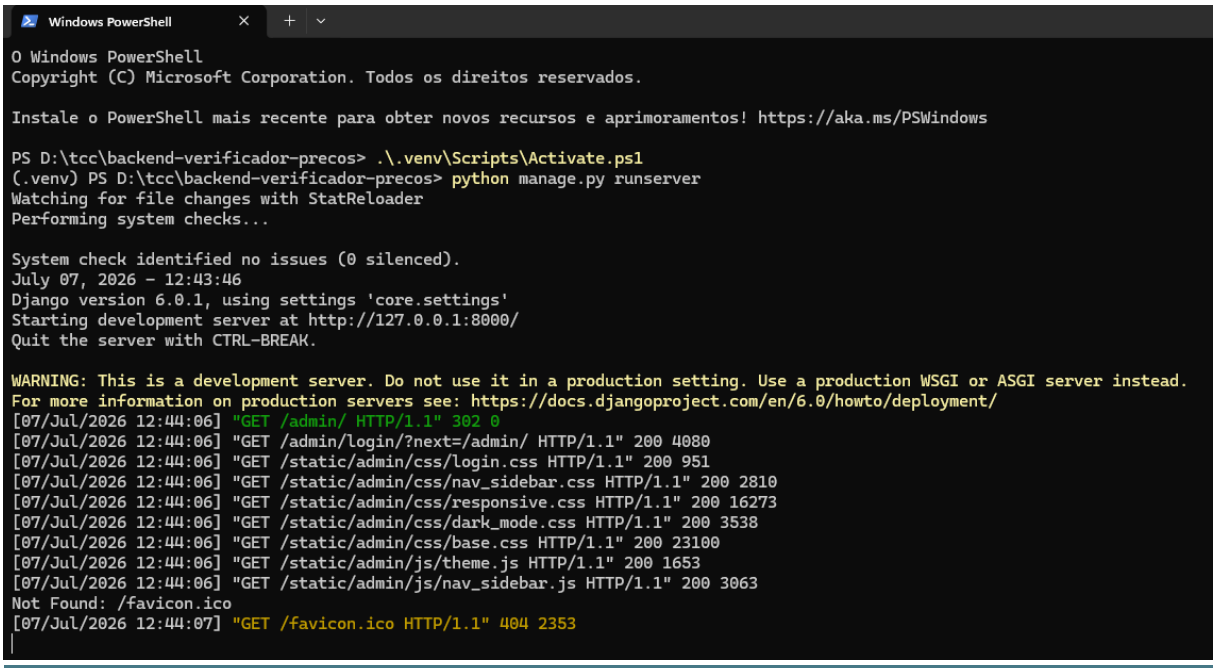
Ex.: Core



Após a criação do projeto e do aplicativo, o servidor de desenvolvimento do Django pode ser iniciado através do comando:

“python manage.py runserver”

Podemos ver que a inicialização do servidor foi bem-sucedida, acessando a página inicial do Django através do endereço: http://127.0.0.1:8000/



Com isso, podemos concluir que o ambiente de desenvolvimento foi configurado corretamente, estando pronto para a modelagem do banco de dados e implementação da API REST.

Configuração e Integração do Banco de Dados MySQL ao Backend Django.

1. Instalação do Driver de conexão entre Django e o servidor MySQL.
   “pip install mysql-connector-python”

2. Arquivo **“settings.py”** modificado para utilizar o MySQL.
   Configuração atual.

```
DATABASES = {
    'default': {
        'ENGINE': 'mysql.connector.django',
        'NAME': os.environ['DB_NAME'],
        'USER': os.environ['DB_USER'],
        'PASSWORD': os.environ['DB_PASSWORD'],
        'HOST': os.environ['DB_HOST'],
        'PORT': os.environ['DB_PORT'],
    }
}
```

3. Aplicação das “Migrations” no MySQL. Após configurar o banco de dados, foram executados os comandos:
   “python manage.py makemigrations” Para verificar se existiam alterações estruturais nas Models do sistema.
   “python manage.py migrate” Para aplicar as migrations padrão do Django no banco MySQL, assim criando automaticamente as tabelas necessárias para:
   - Autenticação de usuários (auth)
   - Administração (admin)
   - Controle de sessões (sessions)
   - Controle interno de tipos de conteúdo (contenttypes)

---

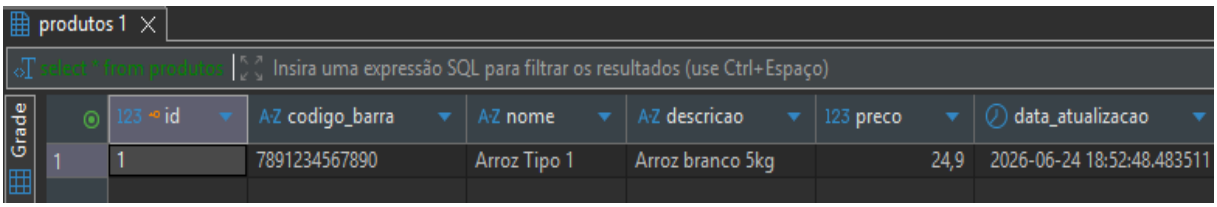
4. Classe produto criada e moldada em “produtos/models.py”
    - **Produto** é a entidade principal do sistema.
    - **codigo_barra** é o identificador usado pelo aplicativo após a leitura da câmera.
    - **nome**, **descricao** e **preco** são as informações retornadas ao usuário.
    - **data_atualizacao** registra quando o produto foi alterado pela última vez.

    A classe herda de **models.Model** para que o Django transforme automaticamente a estrutura em uma tabela do MySQL.

Migrações do Banco de Dados.

Após a criação da classe Produto no arquivo “models.py”, foi necessário gerar a estrutura no banco de dados, utilizando os comandos: “**makemigrations”** e “**migrate”.**

Basicamente, o comando **“makemigrations”** pega a estrutura da classe Produto criada no Django e gera as instruções necessárias para criar a tabela correspondente no banco de dados. Já o comando **“migrate”** executa essas instruções, criando efetivamente a tabela no MySQL.



5. **Configuração da interface administrativa do sistema.**

Após a criação da entidade Produto e das tabelas no banco de dados, foi realizada a configuração do painel administrativo do Django.

O Django já vem com uma interface administrativa nativa que permite gerenciar os dados do sistema através do navegador web, sem a necessidade de criar telas / páginas web específicas para a manutenção dos registros.

Para disponibilizar a entidade Produto no painel administrativo, foi realizado o registro da classe Produto no arquivo “admin.py” do aplicativo produtos.

Exemplo de configuração.

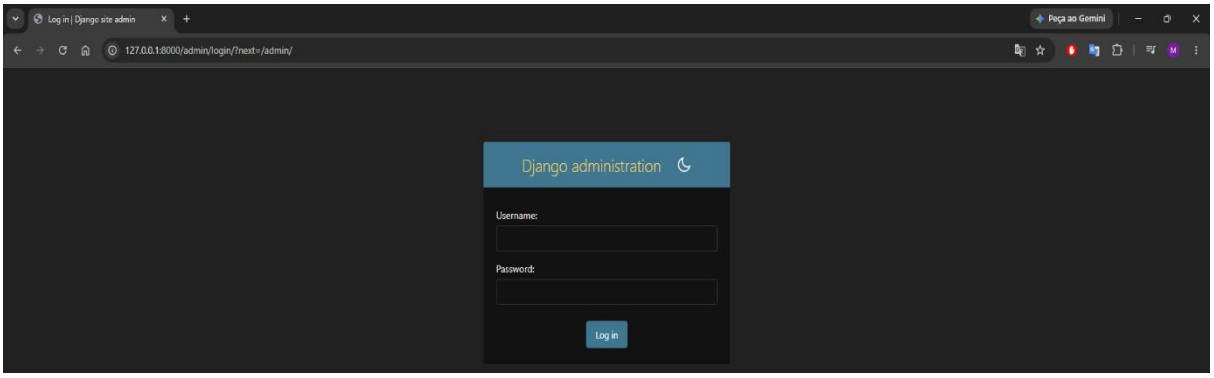
```
from django.contrib import admin
from .models import Produto


@admin.register(Produto)
class ProdutoAdmin(admin.ModelAdmin):
    list_display = ("nome", "codigo_barra", "preco", "data_atualizacao")
    search_fields = ("nome", "codigo_barra")
    list_filter = ("data_atualizacao",)
```

- **“list_display”** Define as informações do produto a exibir.
- **“search_fields”** Permite pesquisas pelos campos definidos. Assim facilitando a localização dos produtos cadastrados.
- **“list_filter”** Adiciona filtros na interface administrativa, permitindo organizar os registros de acordo com a data de atualização.

Após a configuração do painel administrativo, foi criado um usuário administrador, através do comando: **“python manage.py createsuperuser”.**

A partir dessa interface será possível realizar a manutenção dos registros dos produtos. (cadastro, edição, consulta e remoção) diretamente no banco de dados, facilitando os testes e a administração das informações utilizadas pelo aplicativo.



6. Implementação da API REST para Consulta de Produtos.

Após a modelagem do banco de dados e a configuração do painel administrativo, foi iniciada a implementação da API REST responsável pela comunicação entre o aplicativo Android e o servidor.

A função desta API é receber um código de barras enviado pelo aplicativo, consultar as informações do produto no banco de dados MySQL e retornar os dados encontrados no formato JSON.

Essa arquitetura permite que o aplicativo móvel permaneça leve, realizando apenas a leitura do código de barras e a exibição dos resultados, enquanto toda a lógica de consulta fica centralizada no servidor.

Estrutura Implementada

Para a construção da API REST foram criados e configurados os seguintes arquivos:

- produtos/serializers.py
- produtos/views.py
- produtos/urls.py
- core/urls.py

Arquivo serializers.py

O arquivo serializers.py é responsável por converter objetos Python em JSON e também realizar o processo inverso quando necessário.

No contexto deste projeto, ele foi utilizado para transformar os dados do modelo Produto em uma resposta JSON que pode ser interpretada pelo aplicativo Android.

Foi criada a classe:

class ProdutoSerializer(serializers.ModelSerializer):

Essa classe utiliza o ModelSerializer do Django REST Framework, que gera automaticamente a estrutura JSON com base nos campos definidos no modelo Produto.

Os campos disponibilizados para a API foram:

- codigo_barra
- nome
- descricao
- preco
- data_atualizacao

Exemplo de resposta produzida pelo serializer:

{

"codigo_barra": "7891234567890",

"nome": "Arroz Tipo 1",

"descricao": "Arroz branco 5kg",

"preco": "24.90",

"data_atualizacao": "2026-06-24T18:00:00Z"

}

O serializer funciona como uma camada intermediária entre o banco de dados e a API.

Arquivo views.py

O arquivo views.py contém a lógica responsável por atender as requisições recebidas pela API.

Foi criada a classe:

class ProdutoPorCodigoAPIView(RetrieveAPIView):

Essa classe herda de RetrieveAPIView, um recurso do Django REST Framework destinado à consulta de um único registro.

Sua função é:

1. Receber o código de barras enviado na URL.
2. Consultar o banco de dados.
3. Localizar o produto correspondente.
4. Utilizar o serializer para converter o resultado em JSON.
5. Retornar a resposta ao cliente.

Caso o produto exista, os dados são retornados.

Caso não exista, a API retorna automaticamente o erro HTTP 404 (Não Encontrado).

Arquivo produtos/urls.py

O arquivo urls.py do aplicativo produtos define quais endereços estarão disponíveis para consulta.

Foi criado o endpoint:

```
path(
    "produtos/<str:codigo_barra>/",
    ProdutoPorCodigoAPIView.as_view(),
    name="produto-por-codigo"
)
```

Nesse endereço:

produtos/<codigo_barra>/

o valor informado após "produtos/" é interpretado como o código de barras que será utilizado para realizar a busca no banco de dados.

Exemplo:

/api/produtos/7891234567890/

Nesse caso o sistema procura um produto cujo código de barras seja:

"7891234567890".

Arquivo core/urls.py

O arquivo core/urls.py é o ponto central de roteamento do projeto Django.

Sua função é reunir as rotas de todos os aplicativos existentes no sistema.

Foi adicionada a seguinte configuração:

path("api/", include("produtos.urls"))

Com isso, todas as rotas definidas dentro do aplicativo produtos passaram a ficar disponíveis através do prefixo:

/api/

Essa organização facilita a manutenção do sistema e segue as boas práticas de desenvolvimento com Django.

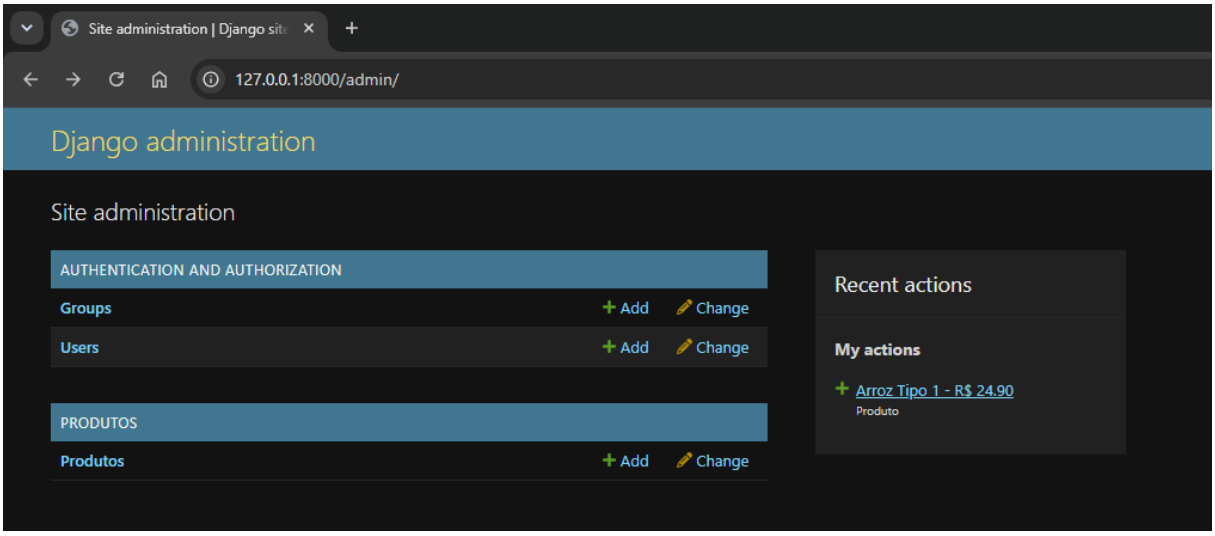
Testes da API

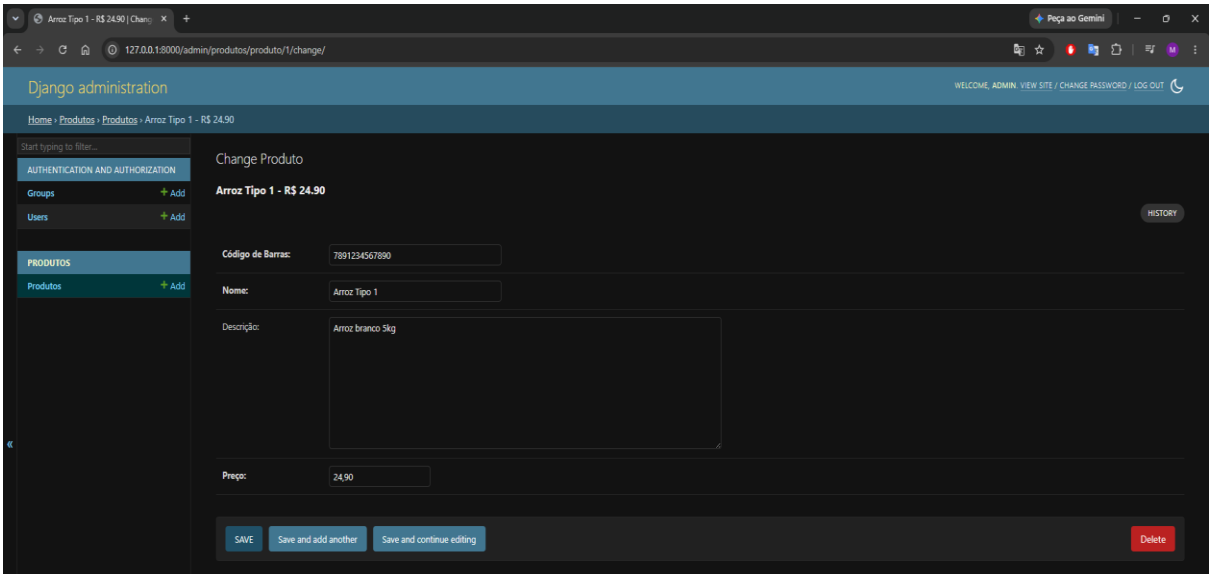
Após a implementação da API, foram realizados testes utilizando o navegador web.

Inicialmente foi efetuado o cadastro manual de produtos através do painel administrativo do Django.

Endereço utilizado:

http://127.0.0.1:8000/admin



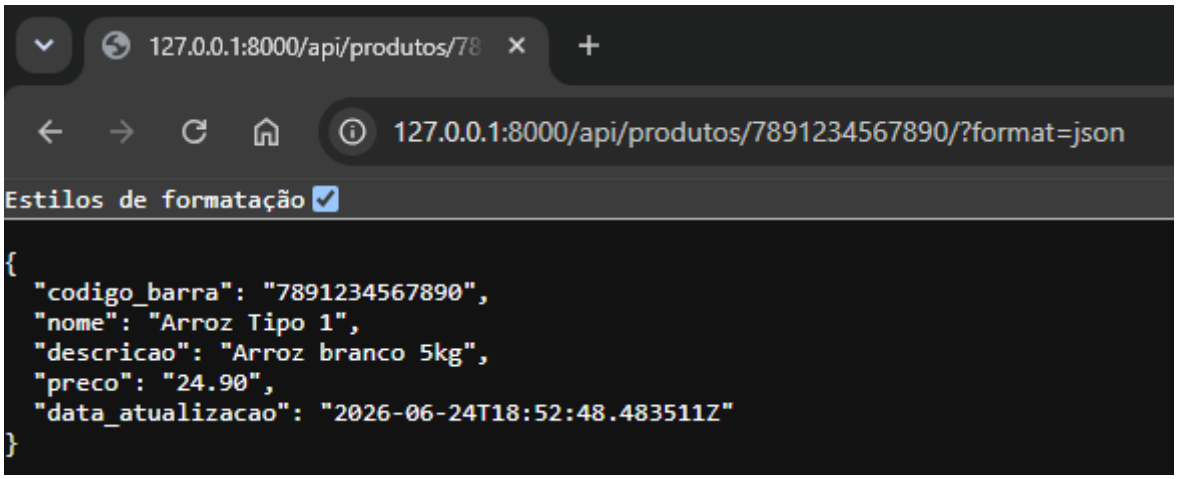


Após o cadastro dos produtos, foi realizada uma requisição diretamente ao endpoint da API.

Exemplo:

http://127.0.0.1:8000/api/produtos/7891234567890/

O sistema retornou corretamente os dados cadastrados no banco de dados no formato **JSON**.



Esse resultado confirmou o correto funcionamento da integração entre:

- Banco de dados MySQL;
- Django ORM;
- Django REST Framework;
- Endpoint REST;
- Serialização dos dados em JSON.

Com isso, a infraestrutura necessária para que o aplicativo Android realize consultas de preços através da rede Wi-Fi foi concluída com sucesso.

Verificação de logs.

Aqui verificamos e analisamos os logs de execução.

- System check identified no issues (0 silenced).
  O Django verificou a configuração do projeto e não encontrou problemas.
- Django version 6.0.1, using settings 'core.settings'
  O projeto está usando corretamente o arquivo de configuração core.settings.
- Starting development server at http://127.0.0.1:8000/
  O servidor de desenvolvimento iniciou normalmente.
- GET /api/produtos/7891234567890/ HTTP/1.1" 200
  Aqui está a execução da consulta: a rota da API respondeu com HTTP 200, Consulta foi processada com sucesso.
- GET /api/produtos/7891234567890/?format=json HTTP/1.1" 200 150
  Mesma consulta também respondeu corretamente no formato JSON.

Tratamento de Erros na Consulta de Produtos

Após a implementação e os testes iniciais da API REST, foi adicionado o tratamento para consultas de produtos inexistentes.

Para isso, o método **“retrieve()”** da classe **“ProdutoPorCodigoAPIView”** foi personalizado, no arquivo **produtos/views.py.**

O código de barras que é recebido através da URL, é utilizado para realizar a consulta no banco de dados:

“Produto.objects.get(codigo_barra=codigo_barra)”

A consulta foi colocada dentro de uma estrutura **“try/except”**. Caso o produto seja encontrado, os dados são enviados ao Serializer e retornados normalmente pela API com o código HTTP 200 OK.

Caso o código de barras informado, não seja localizado no banco de dados. O Django gera a exceção **Produto.DoesNotExist**. Essa exceção é então tratada pela API, que retorna o código **HTTP 404 Not Found** e uma mensagem informando que o produto não foi encontrado.

Exemplo de resposta:

{

"erro": "Produto não encontrado."

}

Com esse tratamento, o aplicativo Android poderá identificar quando uma consulta foi realizada com sucesso ou quando o código de barras lido não está cadastrado no sistema.

Foram realizados dois testes após a alteração:

- Utilizando um código de barras existente: Retornou HTTP 200 e Json com as informações do produto.
- Utilizando um código inexistente: HTTP 404 e a mensagem de erro definida.

Validação do Código de Barras

Após a implementação do tratamento para produtos não encontrados, foi adicionada uma validação do código de barras recebido pela API.

Para a organização do projeto, foi criado o arquivo **“produtos/validador.py”**, responsável por concentrar a lógica de validação.

Foi definida uma função que verifica se o código recebido possui somente números e se o tamanho está dentro do limite padrão de códigos de barras.

A validação é realizada antes da consulta ao banco de dados. Dessa forma, valores que não possuem um formato aceito são rejeitados sem a necessidade de realizar uma consulta no MySQL.

Quando um código inválido é recebido, a API retorna o código HTTP 400 Bad Request com a mensagem:

{

"erro": "Código de barras inválido."

}

Foram realizados testes e confirmado o comportamento esperado da API:

- **HTTP 200:** Produto encontrado;
- **HTTP 404:** Código válido, porém produto não cadastrado;
- **HTTP 400:** Código de barras com formato inválido.

Documentação da API com OpenAPI e Swagger

Após a implementação do tratamento de erros e da validação dos códigos de barras, foi adicionada uma ferramenta para documentar de forma padronizada os endpoints da API REST.

Para essa finalidade foi utilizada a biblioteca: **drf-spectacular 0.30.0**

“python -m pip install drf-spectacular”

A biblioteca foi adicionada ao arquivo: **“core/settings.py”**

Também foi configurada como responsável pela geração do schema OpenAPI utilizado pelo Django REST Framework.

A configuração adicionada foi:

```
REST_FRAMEWORK = {
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}
```

Também foram adicionadas informações gerais da API:

```
SPECTACULAR_SETTINGS = {
    'TITLE': 'API Verificador de Preços',
    'DESCRIPTION': 'API REST responsável pela consulta de produtos através do código de barras.',
    'VERSION': '1.0.0',
    'SERVE_INCLUDE_SCHEMA': False,
}
```

Essas informações são utilizadas para identificar a API dentro da documentação gerada.

OpenAPI

O OpenAPI é utilizado para descrever de maneira padronizada a estrutura da API.

Através dele podem ser documentados:

- endpoints;
- métodos HTTP;
- parâmetros;
- estrutura dos dados;
- possíveis respostas;
- códigos HTTP retornados.

Foi criada a rota: **“/api/schema/”**

O endereço completo durante o desenvolvimento é:
http://127.0.0.1:8000/api/schema/

Ao acessar essa rota, o sistema disponibiliza a especificação OpenAPI da aplicação em formato YAML.

Também é possível gerar e validar o schema através do terminal utilizando:

python manage.py spectacular --file schema.yml –validate

Esse comando gera o arquivo: “schema.yml” na pasta principal do projeto.

Swagger UI

Além do schema OpenAPI, também foi configurada a interface Swagger UI.

A interface pode ser acessada através do endereço: http://127.0.0.1:8000/api/docs/

O Swagger utiliza o schema OpenAPI e apresenta os endpoints através de uma interface visual.

Por meio dessa interface é possível:

- visualizar os endpoints existentes;
- visualizar os parâmetros necessários;
- verificar os tipos de respostas;
- executar requisições;
- visualizar os códigos HTTP retornados;
- testar a API diretamente pelo navegador.

O Swagger é utilizado como ferramenta de documentação e teste da API e não participa diretamente da comunicação entre o futuro aplicativo Android e o backend.

Documentação do Endpoint de Produtos

O endpoint principal do sistema foi documentado como:

GET /api/produtos/{codigo_barra}/

O parâmetro: “codigo_barra” representa o código utilizado para localizar o produto no banco de dados.

Na documentação OpenAPI foram definidos os seguintes retornos:

- **HTTP 200 OK**

  Indica que o produto foi localizado corretamente.

  Exemplo:

  {

  "codigo_barra": "7891234567890",

  "nome": "Arroz Tipo 1",

  "descricao": "Arroz branco 5kg",

  "preco": "24.90",

  "data_atualizacao": "..."

  }

- **HTTP 400 Bad Request**

  Indica que o código informado possui um formato inválido.

  Exemplo:

  {

  "erro": "Código de barras inválido."

  }

- **HTTP 404 Not Found**

  Indica que o código possui um formato aceito, porém não existe produto correspondente cadastrado no banco de dados.

  Exemplo:

  {

  "erro": "Produto não encontrado."

  }

Serializer para Respostas de Erro

Para permitir que o OpenAPI documentasse corretamente a estrutura das mensagens de erro, foi criado no arquivo: **produtos/serializers.py** o ErroSerializer.

Sua função é definir que as respostas de erro possuem um campo denominado: “erro” contendo uma mensagem de texto.

Exemplo:

```
class ErroSerializer(serializers.Serializer):
    erro = serializers.CharField()
```

Esse Serializer não representa uma tabela no banco de dados. Sua função é somente definir a estrutura da resposta utilizada pela API.

Documentação da View com extend_schema

No arquivo: **“produtos/views.py”** foi utilizado o decorator: @extend_schema

O objetivo dessa configuração é informar ao OpenAPI detalhes que não seriam identificados automaticamente.

Foram documentados:

- nome da operação;
- descrição da consulta;
- parâmetro codigo_barra;
- resposta HTTP 200;
- resposta HTTP 400;
- resposta HTTP 404.

Dessa forma, o Swagger consegue apresentar de maneira clara o funcionamento do endpoint de consulta de produtos.

Configuração das Rotas OpenAPI e Swagger

No arquivo: **“core/urls.py”** foram adicionadas duas novas rotas.

A primeira disponibiliza o schema OpenAPI:

/api/schema/

A segunda disponibiliza a interface Swagger:

/api/docs/

Com isso, o sistema passou a possuir três acessos principais relacionados à API:

/api/produtos/{codigo_barra}/

Endpoint utilizado para realizar a consulta do produto.

/api/schema/

Disponibiliza a especificação OpenAPI.

/api/docs/

Disponibiliza a documentação visual através do Swagger UI.

Testes Realizados

Após a implementação das novas funcionalidades, foram realizados testes para verificar o funcionamento completo do backend.

Foram verificados:

- ambiente virtual Python;
- dependências instaladas;
- configuração do Django;
- migrações;
- conexão com o banco de dados MySQL;
- funcionamento do Django ORM;
- validação do código de barras;
- consulta de produto existente;
- consulta de produto inexistente;
- consulta utilizando código inválido;
- geração do schema OpenAPI;
- validação do schema;
- acesso ao Swagger UI.

Os principais resultados esperados foram confirmados:

“HTTP 200 OK” - Produto encontrado corretamente.

“HTTP 400 Bad Request” - Código de barras inválido.

“HTTP 404 Not Found” - Produto não cadastrado.

Também foi confirmado o funcionamento das rotas: **“/api/schema/”** e **“/api/docs/”**

Com isso, o backend passou a possuir tratamento de erros, validação dos dados recebidos e documentação padronizada da API.

Estado Atual do Backend

Após as implementações realizadas, o backend possui atualmente:

- Django 6.0.1;
- Django REST Framework 3.16.1;
- MySQL;
- Django ORM;
- cadastro e manutenção de produtos através do Django Admin;
- consulta de produtos por código de barras;
- retorno das informações em JSON;
- tratamento para produtos não encontrados;
- validação do código de barras;
- respostas HTTP 200, 400 e 404;
- documentação OpenAPI;
- interface Swagger UI para visualização e testes da API.

Com essas etapas concluídas, a estrutura principal do backend encontra-se funcional e documentada, ficando preparada para as próximas etapas de testes automatizados e integração com o aplicativo Android.

Implementação de Testes Automatizados do Backend

Após a implementação e os testes manuais da API REST, foram adicionados testes automatizados com o objetivo de verificar se as principais funcionalidades do backend continuam funcionando corretamente após alterações no código.

Os testes foram implementados no arquivo: **“produtos/tests.py”**

Para a realização dos testes foi utilizada a classe **APITestCase**, disponibilizada pelo Django REST Framework e destinada ao teste de APIs.

Antes de cada teste é criado um produto temporário através do método **setUp()**.

Esse produto é utilizado apenas durante a execução dos testes e não interfere nos dados cadastrados no banco principal do sistema.

Foram implementados quatro testes principais.

Consulta de Produto Existente.

O primeiro teste realiza uma requisição utilizando o código de barras de um produto existente.

O resultado esperado é: HTTP 200 OK

Além do código HTTP, o teste verifica se o código de barras e o nome retornados pela API correspondem aos dados do produto utilizado no teste.

Consulta de Produto Não Cadastrado.

O segundo teste utiliza um código de barras com formato válido, porém sem produto correspondente cadastrado.

O resultado esperado é: HTTP 404 Not Found

Também é verificado se a API retorna a mensagem: “Produto não encontrado.”

Consulta Utilizando Código Inválido.

O terceiro teste utiliza um código contendo letras.

O resultado esperado é: HTTP 400 Bad Request

Também é verificado se a mensagem retornada é: “Código de barras inválido.”

Código Acima do Limite Definido.

O quarto teste utiliza um código com tamanho superior ao limite atualmente definido pelo sistema. “20 caracteres”.

Nesse caso também é esperado: “HTTP 400 Bad Request”

Com a mensagem indicando que o código de barras é inválido.

Execução dos Testes.

Os testes podem ser executados através do comando:

“python manage.py test produtos”

Durante a execução, o Django utiliza um banco de dados separado para os testes, evitando alterações no banco utilizado pelo sistema durante o desenvolvimento.

Para exibir informações mais detalhadas sobre cada teste pode ser utilizado:

“python manage.py test produtos --verbosity 2”

Os quatro testes foram executados com sucesso, confirmando o comportamento esperado para consultas válidas, produtos inexistentes e códigos de barras inválidos.

A utilização de testes automatizados facilita a identificação de possíveis falhas após futuras alterações no sistema e reduz a necessidade de repetir manualmente todos os testes da API.

Ampliação dos Testes Automatizados

Após a implementação inicial dos quatro testes automatizados, foram adicionados mais dois testes com o objetivo de aumentar a verificação do comportamento da API.

O primeiro teste adicional verifica o conteúdo completo retornado durante uma consulta de produto realizada com sucesso.

Além do código HTTP 200 OK, passaram a ser verificados os seguintes campos:

- código de barras;
- nome;
- descrição;
- preço;
- data de atualização.

Esse teste ajuda a garantir que a estrutura da resposta utilizada futuramente pelo aplicativo Android continue sendo retornada corretamente mesmo após alterações no backend.

Também foi criado um teste para a documentação OpenAPI.

Esse teste acessa a rota responsável pelo schema da API e verifica se ela retorna HTTP 200 OK.

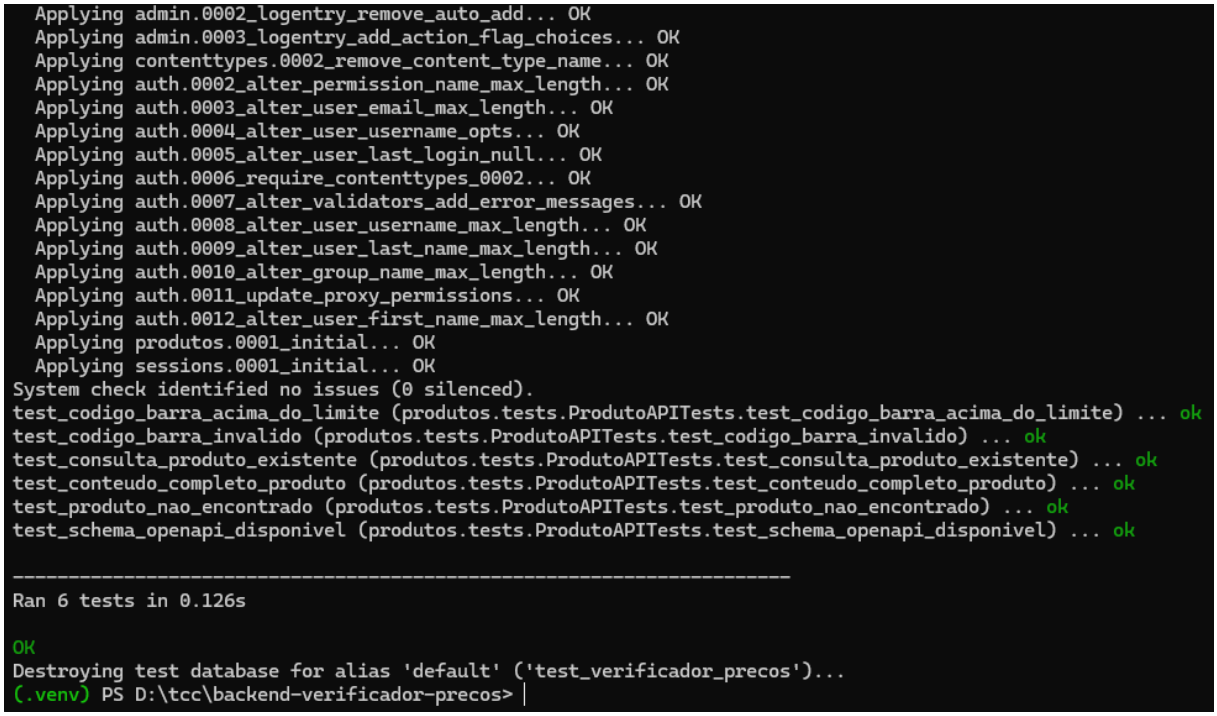
Além disso, é confirmado que o endpoint: “/api/produtos/{codigo_barra}/” está presente na especificação gerada.

Com essas alterações, a suíte passou de quatro para seis testes automatizados, abrangendo tanto o funcionamento da consulta de produtos quanto a estrutura da resposta e a documentação da API.

Os testes podem ser executados através do comando:

“python manage.py test produtos --verbosity 2”

A execução dos seis testes permite verificar automaticamente as principais funcionalidades atualmente implementadas no backend.



Utilização de Variáveis de Ambiente para Configuração do Banco de Dados.

Inicialmente as informações utilizadas para conexão com o banco de dados MySQL foram definidas diretamente no arquivo “settings.py”.

Para evitar que informações sensíveis permanecessem diretamente no código-fonte, foi implementado o uso de variáveis de ambiente.

Para realizar o carregamento dessas configurações foi utilizada a biblioteca:

“python-dotenv”.

A instalação foi realizada através do comando:

python -m pip install python-dotenv

Na raiz do projeto foi criado o arquivo “.env”

Esse arquivo passou a armazenar as informações necessárias para conexão com o banco de dados:

DB_NAME

DB_USER

DB_PASSWORD

DB_HOST

DB_PORT

No arquivo “settings.py”, esses valores passaram a ser obtidos através da função “os.getenv()”.

Dessa forma, a configuração do banco de dados deixou de armazenar diretamente dados como usuário e senha.

O arquivo “.env” também foi adicionado ao “.gitignore”, evitando que as credenciais sejam incluídas no repositório do projeto.

Além disso, foi criado o arquivo “.env.example”, contendo apenas exemplos das variáveis necessárias, sem apresentar dados sensíveis. Esse arquivo serve como referência para a configuração do projeto em outros ambientes.

Após a alteração, foram realizados testes de conexão com o banco de dados, inicialização do servidor e execução da suíte de testes automatizados. Todos os testes foram executados com sucesso. Essa alteração melhora a organização e a segurança do projeto ao separar as configurações específicas do ambiente das informações armazenadas no código-fonte.

Teste de Acesso à API pela Rede Local

Após a configuração do backend, foi realizado um teste de acesso à API por meio de um celular conectado à mesma rede Wi-Fi do servidor Django. Neste caso um notebook.

Inicialmente, o servidor Django era acessado apenas localmente: http://127.0.0.1:8000/

Para permitir o acesso por outros dispositivos, o servidor passou a ser iniciado com o comando: “python manage.py runserver 0.0.0.0:8000”

O endereço 0.0.0.0 permite que o Django aceite conexões pelas interfaces de rede disponíveis no computador.

Durante o teste, o servidor “notebook” utilizava o endereço: 192.168.100.4 e o celular 192.168.100.7

O endpoint foi acessado pelo celular através do endereço: http://192.168.100.4:8000/api/produtos/7891234567890/?format=json

A requisição retornou: “HTTP 200 OK” confirmando que o celular conseguiu acessar corretamente a API Django pela rede local.

Funcionamento da Comunicação

Para facilitar a compreensão da arquitetura utilizada no teste, pode-se utilizar a seguinte comparação:

- Celular = cliente entrando em uma loja
- Notebook = prédio da loja
- Porta 8000 = porta de entrada
- Django = funcionário que atende o cliente
- MySQL = estoque da loja

O fluxo da comunicação pode ser representado da seguinte forma:

```
Celular
192.168.100.7
        ↓
Rede Wi-Fi
        ↓
Notebook
192.168.100.4
        ↓
Porta 8000
        ↓
Django REST Framework
        ↓
Django ORM
        ↓
MySQL
        ↓
Resposta JSON
        ↓
Celular
```

O celular atua como cliente, enquanto o notebook executa o backend Django e disponibiliza o serviço através da porta 8000.

Configuração de ALLOWED_HOSTS

Para permitir o acesso utilizando o endereço IP do notebook, foi necessária a configuração do ALLOWED_HOSTS.

Como esse endereço pode variar conforme a rede utilizada, a configuração foi transferida para o arquivo “.env”.

Exemplo: DJANGO_ALLOWED_HOSTS=127.0.0.1,localhost,192.168.100.4

No arquivo **“settings.py”**, os valores são carregados a partir da variável de ambiente:

```
ALLOWED_HOSTS = [
    host.strip()
    for host in os.environ['DJANGO_ALLOWED_HOSTS'].split(',')
    if host.strip()
]
```

Dessa forma, caso o endereço IP do notebook seja alterado, basta modificar o arquivo “.env”, sem necessidade de alterar o código-fonte.

Porta de Comunicação: A porta utilizada durante o desenvolvimento permaneceu como: 8000

Ela é definida no momento em que o servidor é iniciado: **python manage.py runserver 0.0.0.0:8000**

Aplicativo Android

No ambiente de desenvolvimento, o aplicativo Android utilizará uma URL semelhante a: http://192.168.100.4:8000/

Em produção, esse endereço será substituído por um domínio com HTTPS, por exemplo: https://api.verificadorprecos.com/

Para evitar que a URL fique definida diretamente no código Kotlin, a configuração será separada utilizando recursos do Gradle e BuildConfig.

Com esse teste, foi confirmada a comunicação entre um dispositivo móvel e o backend Django pela rede local.

Configuração Inicial do Ambiente Android

Após a conclusão da etapa principal do backend, foi iniciado o desenvolvimento do aplicativo Android responsável pela consulta dos produtos.

Para isso, foi instalado o **Android Studio**, ambiente de desenvolvimento utilizado para criação, compilação e execução de aplicações Android.

Durante a configuração inicial foi utilizada a instalação padrão do Android Studio, incluindo os componentes necessários do Android SDK e das ferramentas de compilação.

Criação do Projeto Android:

Foi criado um novo projeto Android com as seguintes características:

- Nome: VerificadorPrecos
- Linguagem: Kotlin
- Interface: Jetpack Compose

O projeto foi organizado separadamente do backend Django, mantendo a divisão entre: **“backend-verificador-precos” / “android-verificador-precos”**

Essa separação facilita a organização do sistema, pois o backend e o aplicativo Android possuem responsabilidades diferentes.

O arquivo principal criado pelo Android Studio foi: **MainActivity.kt**

Esse arquivo contém a Activity inicial da aplicação e utiliza o **Jetpack Compose** para construção da interface gráfica.

Gradle

O projeto utiliza o **Gradle** como ferramenta de automação da compilação.

O Gradle é responsável por tarefas como:

- compilar o código Kotlin
- gerenciar dependências
- gerar o APK
- preparar o aplicativo para instalação

Após a criação do projeto, foi realizada a sincronização dos arquivos Gradle e o ambiente foi carregado corretamente pelo Android Studio.

Utilização de Dispositivo Físico

Devido ao objetivo do projeto e também para reduzir o consumo de recursos do computador, os testes foram realizados diretamente em um dispositivo Android físico.

Foi utilizado um **Motorola Moto G22**, conectado ao notebook através de USB.

No aparelho foi habilitada a opção: **“Depuração USB”**

Primeiro Build e Execução

Após a configuração do projeto, foi realizado o primeiro processo de compilação.

O Android Studio utilizou o Gradle para gerar o aplicativo e, através do ADB, realizou a instalação no dispositivo conectado.

O processo foi concluído com sucesso, apresentando as mensagens:

“Gradle build finished” e **“Install successfully finished”**

Após a instalação, o aplicativo foi aberto automaticamente no celular.

A tela inicial exibiu a mensagem **“Hello Android!”** confirmando que o código padrão criado pelo Android Studio foi executado corretamente.

O aplicativo também passou a aparecer normalmente na lista de aplicativos instalados no celular com o nome: **“VerificadorPrecos”**

Resultado do Teste

O primeiro teste confirmou o funcionamento correto dos principais componentes do ambiente Android:

- Android Studio
- Kotlin
- Jetpack Compose
- Gradle
- ADB
- Compilação do projeto
- Geração do APK
- Instalação no dispositivo
- Execução do aplicativo

Com essa etapa concluída, o ambiente de desenvolvimento Android encontra-se configurado e funcional.

Desenvolvimento da Interface Inicial do Aplicativo Android

Após a configuração do ambiente Android e a execução do primeiro aplicativo de teste no dispositivo físico, foi iniciada a criação da interface do sistema **Verificador de Preços**.

A tela inicial foi desenvolvida utilizando **Kotlin** e **Jetpack Compose** e passou a possuir os seguintes elementos:

Título: Verificador de Preços

Campo para informar o código de barras: **“Tipo: OutlinedTextField”**

Botão: CONSULTAR

Campos para exibição dos dados:

- Produto:
- Descrição:
- Preço:

O objetivo dessa primeira interface foi preparar a estrutura visual do aplicativo antes da implementação da comunicação com o backend.

Para armazenar o valor digitado pelo usuário foi utilizado um estado do Jetpack Compose:

```
var codigoBarra by remember {
    mutableStateOf("")
}
```

Dessa forma, sempre que o usuário digita um código, o valor informado é armazenado na variável **codigoBarra**.

A cada nova implantação era realizado um novo teste, para validar cada passo.

Configuração da Comunicação com o Backend

Após a validação da interface, foi iniciada a configuração necessária para permitir a comunicação entre o aplicativo Android e a API REST desenvolvida em Django.

Foram adicionadas ao projeto Android as bibliotecas responsáveis pela comunicação HTTP, conversão dos dados JSON e utilização da arquitetura MVVM.

Foram utilizadas:

- Retrofit
- Retrofit Moshi Converter
- Moshi
- Lifecycle ViewModel

O **Retrofit** será responsável por realizar as requisições HTTP entre o aplicativo Android e o backend Django.

O **Moshi** será responsável pela conversão dos dados JSON recebidos da API para objetos Kotlin utilizados pelo aplicativo.

Também foram adicionadas as dependências do **ViewModel**, utilizadas posteriormente para separar a lógica de consulta da interface gráfica.

O teste foi concluído com sucesso, sem erros de compilação.

Configuração da URL do Backend

Para evitar que o endereço do backend ficasse escrito diretamente em vários arquivos Kotlin, foi utilizada a classe: **“BuildConfig”.**

No arquivo: **“build.gradle.kts”** foi configurado o endereço utilizado durante o desenvolvimento: **http://192.168.100.4:8000/**

A configuração foi disponibilizada através da variável: **“BuildConfig.BASE_URL”**

Dessa forma, as classes responsáveis pela comunicação podem obter o endereço do servidor através dessa configuração.

Essa organização também facilitará uma futura alteração entre o ambiente de desenvolvimento e um possível ambiente de produção.

Permissão de Acesso à Internet

Para permitir que o app Android realize requisições de rede, foi adicionada no arquivo: **AndroidManifest.xml** a permissão: **<uses-permission android:name="android.permission.INTERNET" />**

Essa permissão permite que o aplicativo realize conexões com serviços disponíveis através da rede.

Configuração de HTTP para o Ambiente Local

Durante o desenvolvimento, o backend Django está sendo executado na rede local utilizando HTTP: **http://192.168.100.4:8000/**

Como as versões atuais do Android possuem restrições para conexões HTTP sem criptografia, foi criado o arquivo: **res/xml/network_security_config.xml** com uma configuração permitindo tráfego HTTP durante o desenvolvimento: **<base-config cleartextTrafficPermitted="true" />**

Esta configuração também foi vinculada ao aplicativo no arquivo: **AndroidManifest.xml**

Utilizando: **android:networkSecurityConfig="@xml/network_security_config"**

Essa configuração é utilizada somente no ambiente de desenvolvimento local.

Em um ambiente de produção, o objetivo será utilizar comunicação através de: **HTTPS**

Organização das Classes de Comunicação

Para melhorar a organização do projeto Android, foram criados novos pacotes.

A estrutura principal passou a ser:

```
com.tcc.verificadorprecos

MainActivity.kt

data
    ProdutoDto.kt

network
    ProdutoApi.kt
    RetrofitClient.kt

viewmodel
    ProdutoViewModel.kt
```

Essa separação permite dividir as responsabilidades do aplicativo entre interface, dados, comunicação e lógica.

Classe ProdutoDto

Foi criada a classe: **ProdutoDto.kt** responsável por representar no aplicativo Android os dados do produto retornados pela API Django.

```kotlin
data class ProdutoDto(

    @Json(name = "codigo_barra")
    val codigoBarra: String,

    val nome: String,

    val descricao: String,

    val preco: String,

    @Json(name = "data_atualizacao")
    val dataAtualizacao: String
)
```

A API Django retorna informações semelhantes a:

```json
{
    "codigo_barra": "7891234567890",
    "nome": "Arroz Tipo 1",
    "descricao": "Arroz branco 5kg",
    "preco": "24.90",
    "data_atualizacao": "..."
}
```

O Moshi realiza a conversão dessas informações para um objeto ProdutoDto.

As anotações: **@Json(name = "codigo_barra")** e **@Json(name = "data_atualizacao")** foram utilizadas porque os nomes utilizados pelo JSON seguem o padrão com _, enquanto no Kotlin foram utilizados nomes em camelCase.

O campo: **val preco: String** foi mantido como texto por questões de compatibilidade. Caso no futuro seja necessário efetuar cálculos, o campo será ajustado.

Interface ProdutoApi

Foi criado o arquivo: **ProdutoApi.kt** responsável por representar no Retrofit o endpoint existente no backend.

A interface foi definida como:

```
interface ProdutoApi {

    @GET("api/produtos/{codigo_barra}/")
    suspend fun consultarProduto(
        @Path("codigo_barra") codigoBarra: String
    ): Response<ProdutoDto>
}
```

Essa configuração corresponde ao endpoint desenvolvido anteriormente no Django:

GET /api/produtos/{codigo_barra}/

Quando o aplicativo informa, por exemplo: 7891234567890 o Retrofit monta uma requisição para: http://192.168.100.4:8000/api/produtos/7891234567890/

O parâmetro: **@Path("codigo_barra")** é responsável por inserir o código informado dentro da URL.

A função foi definida como: **suspend** permitindo que a operação de rede seja realizada através de coroutines sem bloquear a interface principal do aplicativo.

Também foi utilizado: **Response<ProdutoDto>** para permitir que o aplicativo analise os códigos HTTP retornados pelo backend, como:

- **HTTP 200**
- **HTTP 400**
- **HTTP 404**

Configuração do Retrofit

Foi criado o arquivo: **RetrofitClient.kt** responsável por centralizar a configuração do Retrofit. Primeiramente foi configurado o Moshi:

```
private val moshi = Moshi.Builder()
    .add(KotlinJsonAdapterFactory())
    .build()
```

Em seguida foi criada a configuração do Retrofit:

```
private val retrofit = Retrofit.Builder()
    .baseUrl(BuildConfig.BASE_URL)
    .addConverterFactory(
        MoshiConverterFactory.create(moshi)
    )
    .build()
```

O endereço principal do backend é obtido através de: **BuildConfig.BASE_URL**

Também foi criada a instância da interface da API:

```
val produtoApi: ProdutoApi =
    retrofit.create(ProdutoApi::class.java)
```

Com isso, a aplicação passou a possuir uma estrutura preparada para realizar requisições HTTP para o servidor Django. Após a criação das classes:

- **ProdutoDto.kt**
- **ProdutoApi.kt**
- **RetrofitClient.kt**

Foi realizada nova compilação e execução do aplicativo.

O teste foi concluído com sucesso.

Implementação do ViewModel

Após a preparação da comunicação com o servidor foi criado o arquivo: **ProdutoViewModel.kt**

O objetivo do ViewModel é manter a lógica de consulta separada da interface gráfica.

A organização utilizada segue o padrão arquitetural MVVM – Model View ViewModel.

De maneira simplificada:

Interface Android => ViewModel => Retrofit => API Django

No ViewModel foi criada uma estrutura de estados chamada: ProdutoUiState

Foram definidos quatro estados principais: Inicial; Carregando; Sucesso; Erro.

O estado “inicial” representa a tela antes de uma consulta.

O estado “Carregando” é utilizado enquanto o aplicativo aguarda a resposta do servidor.

O estado “Sucesso” possui o objeto: **ProdutoDto** com as informações recebidas da API.

O estado “Erro” possui uma mensagem que será exibida ao usuário.

Controle dos Estados com StateFlow

Para armazenar e atualizar o estado da tela foi utilizado: **MutableStateFlow**

A estrutura principal ficou:

```
private val _uiState =
    MutableStateFlow<ProdutoUiState>(ProdutoUiState.Inicial)
```

Para permitir que a interface apenas observe o estado, foi disponibilizado:

```
val uiState: StateFlow<ProdutoUiState> =
    _uiState.asStateFlow()
```

Quando ocorre alguma alteração na consulta, o ViewModel atualiza o estado e o Jetpack Compose atualiza automaticamente a interface.

Consulta de Produtos pelo ViewModel

Dentro do ProdutoViewModel foi criada a função: **consultarProduto()**

Essa função recebe o código digitado pelo usuário e antes de realizar a requisição é verificado se o campo está vazio.

Caso nenhum código tenha sido informado, é retornada a mensagem: Informe um código de barras.

Caso exista um código, o ViewModel utiliza: **viewModelScope.launch** para iniciar a operação de rede. Durante a consulta, o estado passa para: Carregando.

Em seguida é realizada a chamada:
RetrofitClient.produtoApi.consultarProduto(codigo) e A resposta da API é então analisada.

Tratamento das Respostas HTTP no Android

O aplicativo foi configurado para interpretar os mesmos retornos já implementados no backend.

- Quando a API retorna: “HTTP 200 OK” o corpo da resposta é convertido pelo Moshi em um objeto: ProdutoDto, e o estado é alterado para: **Sucesso**.
- Quando a API retorna: “HTTP 400 Bad Request” o aplicativo apresenta:**Código de barras inválido**.
- Quando retorna: “HTTP 404 Not Found” é exibido: **Produto não encontrado**.

Também foi implementado tratamento para problemas de comunicação.

Caso não seja possível acessar o servidor, é exibido: **Não foi possível conectar ao servidor.**

Integração do ViewModel com a Interface

Após a implementação do ViewModel, o arquivo: **MainActivity.kt** foi atualizado.

A função da tela passou a receber:

produtoViewModel: ProdutoViewModel = viewModel()

O estado do ViewModel é observado pelo Compose através de:

val uiState by produtoViewModel.uiState.collectAsState()

O botão: CONSULTAR passou a executar:

produtoViewModel.consultarProduto(codigoBarra)

Dessa forma, o código informado na interface é enviado ao ViewModel, que realiza a chamada à API. Enquanto uma consulta está sendo realizada, a interface apresenta a mensagem: **Consultando produto...** junto de um indicador: **CircularProgressIndicator()**

Durante esse período o botão de consulta fica temporariamente desabilitado, evitando múltiplas chamadas simultâneas.

Exibição dos Dados Recebidos

Quando o estado é: **Sucesso** as informações do **ProdutoDto** são utilizadas diretamente na interface.

Teste Integrado entre Android e Backend

Após a conclusão da implementação foi realizado o primeiro teste integrado completo entre o aplicativo Android, o backend Django e o banco de dados MySQL.

Servidor iniciado: “**python manage.py runserver 0.0.0.0:8000”** aparelho celular e servidor conectados a mesma rede Wi-Fi.

Foram realizados quatro testes principais:

1. **Consulta de produto existente:** foi informado um código cadastrado no banco de dados. A API respondeu com HTTP 200 OK e o aplicativo exibiu corretamente o nome, a descrição e o preço do produto.
2. **Consulta de produto inexistente:** foi informado um código com formato válido, mas sem produto correspondente no banco. O backend retornou HTTP 404 Not Found e o aplicativo apresentou a mensagem “Produto não encontrado.”
3. **Consulta utilizando código inválido:** foi informado um código contendo caracteres não aceitos pela validação existente no backend. A API retornou HTTP 400 Bad Request e o aplicativo apresentou “Código de barras inválido.”
4. **Teste com o servidor desligado:** o servidor Django foi interrompido e uma nova consulta foi realizada. O aplicativo identificou a falha de comunicação e apresentou “Não foi possível conectar ao servidor.”

Todos os testes apresentaram o comportamento esperado.

Resultado da Integração

Com os testes realizados foi confirmada a comunicação completa entre os componentes do sistema.

O fluxo atualmente funcional pode ser representado por:



Com essa etapa concluída, o aplicativo Android já consegue realizar consultas reais ao backend utilizando o código de barras informado manualmente.

Implementação da Câmera e Leitura de Código de Barras

Após a integração entre o aplicativo Android e o backend Django, foi iniciada a implementação da leitura de códigos de barras utilizando a câmera do smartphone.

Para essa etapa foram utilizadas as bibliotecas: **CameraX e ML Kit Barcode Scanning.**

- O **CameraX** foi utilizado para acessar e exibir a imagem da câmera traseira dentro do aplicativo.
- O **ML Kit** foi utilizado para analisar as imagens capturadas e identificar códigos de barras.

Também foi adicionado no arquivo **AndroidManifest.xml** a permissão:

<uses-permission android:name="android.permission.CAMERA" />

Como a câmera é necessária para o funcionamento principal do aplicativo, também foi declarado o recurso de hardware:

<uses-feature

android:name="android.hardware.camera"

android:required="true" />

Visualização da Câmera no Aplicativo

Foi criado o arquivo: **CameraPreview.kt** responsável por exibir a imagem da câmera dentro da interface desenvolvida com Jetpack Compose.

O aplicativo solicita a permissão de câmera ao usuário e, após a autorização, utiliza a câmera traseira do aparelho.

O CameraX foi configurado utilizando o ciclo de vida da aplicação, permitindo que o sistema gerencie corretamente a abertura e o encerramento da câmera.

O teste foi realizado no **Motorola Moto G22**, confirmando a exibição correta da imagem da câmera dentro do aplicativo.

Implementação da Leitura com ML Kit

Após validar o funcionamento da câmera, foi criado o arquivo: **BarcodeAnalyzer.kt** responsável por analisar as imagens recebidas do CameraX.

Foram configurados formatos comuns de códigos de barras utilizados em produtos:

- EAN-13
- EAN-8
- UPC-A
- UPC-E

A análise das imagens é realizada pelo ML Kit. Quando um código é identificado, o valor encontrado é enviado para a interface do aplicativo.

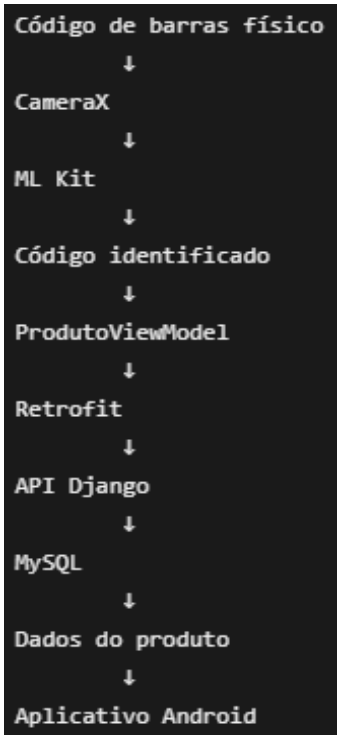
O fluxo implementado passou a ser:

Câmera => CameraX => ImageAnalysis => ML Kit => Código de barras.

Durante os testes, códigos de barras reais foram reconhecidos corretamente e o número identificado passou a preencher automaticamente o campo de código de barras da aplicação.

Integração da Leitura com a Consulta de Produtos

Após confirmar que o ML Kit conseguia identificar corretamente os códigos, a leitura da câmera foi integrada ao **ProdutoViewModel.** Com isso, deixou de ser necessário digitar manualmente o código ou pressionar o botão de consulta após uma leitura. O funcionamento passou a seguir o fluxo:



Após a identificação do código, a consulta é realizada automaticamente.

Caso o produto exista, são exibidos:

- Nome
- Descrição
- Preço

Caso não exista, é apresentada a mensagem correspondente retornada pelo tratamento da aplicação.

Controle de Leituras Repetidas

Durante a leitura pela câmera, o mesmo código pode permanecer visível por vários quadros consecutivos.

Para evitar que isso gerasse várias requisições iguais ao backend, foi implementado um controle de leitura utilizando uma variável de estado.

Após a primeira identificação:

leitura ativa => código detectado => leitura bloqueada => consulta realizada.

Dessa forma, o aplicativo realiza apenas uma consulta para cada ciclo de leitura. Esse comportamento também evita requisições desnecessárias ao servidor Django.

Função Nova Leitura

Foi adicionada a opção: Nova leitura. Essa função permite reiniciar o processo após uma consulta.

Ao selecionar essa opção, o sistema:

- limpa o código de barras
- limpa o resultado exibido
- retorna ao estado inicial
- libera novamente a leitura da câmera

Com isso, o usuário pode consultar diferentes produtos em sequência.

Testes Realizados

Após a conclusão da integração, foram realizados testes no dispositivo físico. Foram verificados:

- abertura da câmera;
- solicitação da permissão;
- exibição da imagem da câmera;
- leitura de código de barras;
- preenchimento automático do código;
- consulta automática ao backend;
- produto existente;
- produto inexistente;
- código inválido;
- bloqueio de consultas repetidas;
- limpeza da tela;
- nova leitura.

Todos os testes apresentaram o comportamento esperado.

Também foi confirmado através dos registros do Django que as requisições chegaram corretamente ao backend e que os códigos HTTP esperados continuaram sendo retornados.

Estado Atual do Aplicativo

Após as implementações realizadas, o aplicativo Android possui atualmente:

- Kotlin;
- Jetpack Compose;
- arquitetura MVVM;
- Retrofit;
- Moshi;
- CameraX;
- ML Kit Barcode Scanning;
- leitura automática de códigos de barras;
- consulta automática à API;
- tratamento dos retornos HTTP;
- exibição dos dados do produto;
- controle contra leituras repetidas;
- função de nova leitura.

Com essa etapa concluída, o fluxo principal previsto para o sistema encontra-se funcional. Atualmente o aplicativo é capaz de identificar um código de barras através da câmera, consultar o backend Django pela rede e apresentar as informações correspondentes ao produto armazenado no banco de dados MySQL.

Melhorias de Usabilidade e Testes do Aplicativo Android

Nesta etapa foram realizados ajustes na interface e na apresentação das informações do aplicativo Android.

O campo de código de barras foi configurado para utilizar **teclado numérico**, facilitando a digitação manual. A leitura automática pela câmera permaneceu funcionando normalmente.

Também foi implementada a formatação do preço utilizando o padrão brasileiro.

- **R$1,99**

O preço também recebeu maior destaque visual na tela.

Após as alterações foram realizados novos testes no dispositivo físico, verificando:

- digitação manual.
- leitura pela câmera.
- consulta ao backend.
- exibição dos dados do produto.
- formatação do preço.
- nova leitura e limpeza da tela.

Todos os testes foram concluídos com sucesso.

Implementação de Repository e Testes Automatizados no Android

Nesta etapa foi realizada uma melhoria na arquitetura do aplicativo Android com a criação da camada **ProdutoRepository**. O objetivo foi reduzir o acoplamento entre o **ProdutoViewModel** e o **RetrofitClient**, facilitando a manutenção e a criação de testes automatizados.

A estrutura passou a seguir o fluxo:

- Interface
- ProdutoViewModel
- ProdutoRepository
- Retrofit
- API Django

Com essa alteração, o **ProdutoViewModel** deixou de acessar diretamente o Retrofit, permitindo que durante os testes seja utilizado um repositório simulado.

Também foram implementados testes unitários utilizando **JUnit** e **kotlinx-coroutines-test**. Os testes são executados localmente, sem necessidade de dispositivo físico, conexão Wi-Fi, servidor Django ou banco MySQL.

Foram criados seis testes para verificar os principais comportamentos do **ProdutoViewModel**:

- Consulta de produto existente;
- Código de barras vazio;
- Erro HTTP 400 - código inválido;
- Erro HTTP 404 - produto não encontrado;
- Falha de conexão com o servidor;
- Retorno ao estado inicial através da função limpar().

Para os testes de comunicação foi utilizado um **ProdutoRepository** simulado, permitindo controlar as respostas retornadas ao **ViewModel**.

Após a implementação, a suíte completa foi executada com sucesso:

- 6 testes executados
- 6 testes aprovados
- BUILD SUCCESSFUL

Com essa etapa, o aplicativo passou a possuir testes automatizados para a lógica principal de consulta de produtos, complementando os testes manuais e integrados realizados anteriormente.

Desenvolvimento e aprimoramento da interface gráfica

1. Objetivo

Esta etapa teve como objetivo reformular a interface gráfica do aplicativo, visando melhorar a organização das informações, a experiência de utilização e a identificação visual do sistema.

O desenvolvimento considerou princípios de Design de Interação, usabilidade e hierarquia visual, especialmente a clareza das informações, a consistência entre telas e o feedback das ações realizadas pelo usuário.

2. Reorganização da Interface

A interface, anteriormente concentrada no arquivo **MainActivity.kt**, foi reorganizada em componentes independentes utilizando Kotlin e Jetpack Compose.

Foram criados os seguintes arquivos:

- **TelaLeitura.kt:** responsável pela apresentação da câmera, moldura de orientação, campo para consulta manual e mensagens de carregamento ou erro.
- **TelaResultado.kt:** responsável pela exibição do nome, descrição, código de barras e preço do produto, além do botão para iniciar uma nova leitura.

O arquivo **MainActivity.kt** passou a coordenar a apresentação dessas telas conforme o estado da consulta, mantendo a integração com o **ProdutoViewModel**.

Essa reorganização favorece a separação de responsabilidades, a legibilidade do código e a manutenção futura do aplicativo.

3. Melhorias Funcionais e de Usabilidade

Foram implementados os seguintes aprimoramentos:

Controle da câmera: a câmera permanece ativa no estado inicial e é retirada da interface durante o processamento da consulta ou após a ocorrência de um erro. A leitura pode ser reativada pelo botão Nova leitura.

Apresentação do resultado: os dados do produto foram organizados em cartões distintos, com destaque para o nome e o preço. Textos extensos podem ocupar várias linhas, e descrições não cadastradas recebem uma mensagem específica.

Tratamento visual dos estados: foram diferenciadas as situações de leitura inicial, consulta em andamento, sucesso e erro, utilizando as informações disponibilizadas pelo **ProdutoUiState**.

Rolagem vertical: as telas receberam suporte à rolagem para facilitar a visualização dos elementos em dispositivos com espaço limitado.

4. Identidade Visual e Tipografia

Foi definida uma identidade visual baseada nas cores verde e branco, associada ao contexto de consulta de produtos no ambiente comercial.

Os arquivos **Color.kt, Theme.kt** e **Type.kt** foram utilizados para centralizar as configurações visuais do aplicativo.

O tema Material Design 3 foi personalizado, substituindo as cores padrão e desativando a aplicação automática das cores dinâmicas do Android. Também foram ajustadas as cores dos cartões e das superfícies para manter a consistência visual.

Na tipografia, foram definidos tamanhos e pesos diferentes conforme a importância das informações, priorizando a legibilidade do nome do produto e o destaque do preço.

5. Testes e Validação

Foram avaliados o funcionamento da leitura automática pela câmera, a consulta manual, o fechamento do teclado, a apresentação de produtos encontrados, o tratamento de códigos inexistentes e o retorno à leitura após consultas e erros.

Também foram realizadas verificações com nomes e descrições extensos, além de inspeção visual das telas inicial, de sucesso e de erro.

Resultado: as etapas implementadas foram compiladas e testadas com sucesso, sem identificação de falhas nos cenários manuais executados.

6. Considerações Finais

A reformulação resultou em uma interface mais organizada, com separação entre os processos de leitura e apresentação dos resultados, identidade visual consistente e melhor comunicação dos estados do sistema.





Implementação de Imagens dos Produtos

1. Objetivo

Esta etapa teve como objetivo implementar a exibição de imagens dos produtos consultados pelo aplicativo, complementando as informações de nome, descrição, código de barras e preço.

A funcionalidade foi desenvolvida preservando a arquitetura já existente. Backend Django, API REST, banco de dados MySQL e o aplicativo Android.

2. Implementação no Backend

O modelo Produto, localizado em **produtos/models.py**, recebeu um campo opcional do tipo **ImageField**, permitindo associar uma imagem a cada produto cadastrado.

Foi utilizada a biblioteca **Pillow 12.3.0** para fornecer o suporte necessário ao processamento e à validação de arquivos de imagem no Django.

O armazenamento foi organizado por meio das configurações **MEDIA_ROOT** e **MEDIA_URL**, definidas em **core/settings.py**. As imagens são armazenadas no diretório **media/img_produtos/**, para organização o nome da imagem deve ser o código de barras “números” do produto que ela representa. O MySQL mantém este valor para referência ao arquivo.

A alteração do banco de dados foi realizada por meio da migração **0002_produto_imagem.py**, Foi adicionada uma nova coluna chamada “**imagem**” do tipo “**varchar(100)”** que armazena o nome da imagem para referencia a mesma.

O envio das fotografias passou a ser realizado pelo próprio painel administrativo do Django, permitindo cadastrar e atualizar imagens dos produtos.

3. Atualização da API REST

O arquivo **produtos/serializers.py** foi atualizado para incluir o campo imagem na resposta JSON da consulta de produtos.

Quando há uma imagem cadastrada, a API retorna seu endereço de acesso. Para produtos sem imagem, o campo pode retornar **null**.

Essa implementação mantém a compatibilidade com as informações anteriores, sem modificar o endpoint de consulta.

Também foi configurada, em **core/urls.py**, a disponibilização dos arquivos de mídia no ambiente de desenvolvimento, permitindo o acesso às imagens pelo aplicativo conectado à mesma rede local “Wi-Fi”.

4. Implementação no Aplicativo Android

O arquivo **ProdutoDto.kt** recebeu o atributo opcional imagem, responsável por armazenar a URL retornada pela API.

Para carregar as imagems, foi incorporada à biblioteca **Coil 3**, utilizando seus módulos de integração com Jetpack Compose e comunicação HTTP.

O componente **TelaResultado.kt** foi atualizado para apresentar a imagem acima das informações do produto.

A implementação contempla três situações:

- **Produto com imagem:** a imagem é carregada e exibida, preservando suas proporções.
- **Produto sem imagem:** as informações são apresentadas normalmente, sem reservar espaço desnecessário.
- **Falha no carregamento:** é apresentada uma indicação de imagem indisponível, sem impedir a visualização dos dados e do preço.

5. Testes e Validação

Foram realizados testes funcionais no backend e no smartphone, incluindo cadastro de imagens pelo Django Admin, acesso aos arquivos pela rede local e retorno do endereço da imagem pela API REST.

No Android, foi validada a exibição das imagens na tela de resultado e a continuidade do funcionamento das consultas.

7. Considerações Finais

A implementação ampliou as informações disponibilizadas pelo App, permitindo a identificação visual dos produtos consultados.

A funcionalidade foi incorporada à arquitetura existente, utilizando recursos do Django e do Jetpack Compose, sem necessidade de alterações significativas nas regras de negócio.





Padronização e Gerenciamento de Imagens dos Produtos

1. Objetivo

Esta etapa teve como objetivo reorganizar o gerenciamento das imagens dos produtos, padronizando seu formato, nome e localização, além de eliminar a necessidade de armazenar o caminho das imagens no banco de dados MySQL.

2. Processamento e armazenamento das imagens

Foi criado o módulo **produtos/imagens.py**, responsável pelo processamento automático das imagens recebidas durante o cadastro ou a edição de produtos.

O módulo realiza a validação do arquivo, a conversão para **WebP**, o redimensionamento quando necessário e a padronização do nome utilizando o código de barras do produto.

As imagens são armazenadas seguindo o padrão:
media/img_produtos/CODIGO_DE_BARRAS.webp

Quando uma nova imagem é enviada para um produto que já possui uma, o arquivo anterior é substituído automaticamente. A conversão é realizada antes da substituição, reduzindo o risco de perda da imagem existente em caso de falha no processamento.

Foram definidos limites de 5 MB por arquivo e resolução máxima de 1200 × 1200 pixels para a imagem processada, preservando suas proporções.

3. Alterações no Django Admin

Foi criado o arquivo **produtos/forms.py**, contendo um formulário personalizado para cadastro e edição de produtos.

O campo **imagem_upload** permite receber imagens sem depender de um campo correspondente no modelo de dados.

O arquivo **produtos/admin.py** foi atualizado para integrar esse formulário ao módulo de processamento de imagens.

O sistema passou a permitir:

- Cadastrar produtos com ou sem imagem.
- Converter automaticamente as imagens enviadas para **WebP.**
- Substituir imagens existentes sem confirmação adicional.
- Editar produtos sem alterar suas imagens.
- Renomear a imagem quando o código de barras do produto for modificado.

4. Configuração do armazenamento

O arquivo **core/settings.py** foi atualizado para utilizar configurações definidas no "**.env"**.

Foram introduzidas as seguintes variáveis:

- IMAGENS_BASE_URL: define o endereço público utilizado para acessar as imagens.
- IMAGENS_STORAGE_ROOT: define o diretório físico de armazenamento acessível ao Django.

A subpasta **img_produtos** permanece padronizada no sistema.

Essa configuração permite utilizar diferentes domínios e endereços IP para disponibilizar imagens, sem modificar o código da API ou do aplicativo Android.

O armazenamento remoto independente ainda dependerá da configuração de um mecanismo de gravação acessível ao backend, como um diretório compartilhado ou serviço específico de upload.

5. Alterações no banco de dados e na API REST

O campo imagem foi removido do modelo Produto, utilizando a migração Django 0003_remove_produto_imagem.py.

A alteração foi aplicada com sucesso, preservando os demais campos da tabela produtos.

O arquivo **produtos/serializers.py** também foi atualizado. A API passou a gerar o endereço da imagem automaticamente, utilizando o código de barras e a configuração **IMAGENS_BASE_URL**.

O campo imagem foi mantido na resposta JSON para preservar a compatibilidade com o aplicativo Android.

Quando não existe uma imagem correspondente ao produto, a API retorna **null**.

Foi implementado ainda um parâmetro de versão na URL, baseado na data de modificação do arquivo, para reduzir problemas com imagens desatualizadas em cache.

6. Testes e validação

Foram criados quatro testes automatizados para o processamento de imagens, abrangendo conversão para WebP, nomeação pelo código de barras, substituição de arquivos e tratamento de entradas inválidas.

Após as alterações, o backend executou **10 testes automatizados, todos aprovados**.

Também foram realizados testes manuais no Django Admin, no navegador e no aplicativo Android.

Foi confirmada a remoção da coluna imagem do MySQL, assim como o funcionamento das consultas pela API REST e o acesso aos arquivos WebP por meio de URLs HTTP.

7. Considerações finais

A refatoração simplificou a estrutura de dados do sistema e padronizou o gerenciamento das imagens dos produtos.

A associação entre produto e imagem passou a utilizar exclusivamente o código de barras, eliminando o armazenamento de caminhos no MySQL e mantendo a compatibilidade com a API REST e o aplicativo Android.

A implementação foi concluída e validada nos cenários de desenvolvimento testados.

Exclusão de Produtos e Gerenciamento de Imagens

1. Objetivo

Esta etapa teve como objetivo implementar a exclusão automática da imagem associada a um produto quando seu cadastro for removido pelo Django Admin, evitando o acúmulo de arquivos sem vínculo com produtos existentes.

Também foi adotada a restrição de exclusões em lotes nas operações administrativas para reduzir o risco de acidentes.

2. Implementação

O módulo **produtos/imagens.py** recebeu a função **excluir_imagem_produto(),** responsável por localizar e remover a imagem utilizando o código de barras do produto.

No arquivo **produtos/admin.py**, foi implementado o método **delete_model(),** que realiza a exclusão individual do produto e agenda a remoção de sua imagem por meio de **transaction.on_commit().**

Esse mecanismo permite executar a exclusão do arquivo somente após a confirmação da transação no banco de dados, evitando remover a imagem caso a operação seja revertida.

Quando o produto não possui imagem, a exclusão ocorre normalmente.

3. Segurança operacional

As ações em lote do Django Admin foram desabilitadas por meio da configuração **actions = None**.

Essa decisão foi adotada como medida preventiva de segurança operacional, reduzindo a possibilidade de exclusões acidentais de vários produtos simultaneamente.

A exclusão individual permanece disponível, exigindo que o administrador selecione e confirme a remoção de cada produto.

Essa restrição não substitui os mecanismos de autenticação e autorização do Django.

4. Testes e validação

Foram implementados testes automatizados para verificar:

- A exclusão de uma imagem existente.
- O tratamento de imagens inexistentes.
- A exclusão individual de produtos com imagem.
- A exclusão individual de produtos sem imagem.
- A desativação das ações em lote no Django Admin.

Os testes utilizam um banco de dados de teste e diretórios temporários, evitando alterações nos registros e arquivos reais.

Os testes individuais, os testes automatizados do backend e as verificações manuais foram realizados e aprovados.

5. Considerações finais

A implementação melhorou a consistência entre os cadastros de produtos e os arquivos de imagens, reduzindo o risco de “arquivos órfãos”.

A desativação das operações em lote acrescentou uma proteção preventiva ao uso do Django Admin.

A funcionalidade foi concluída e validada no ambiente de desenvolvimento. A comunicação com a API REST e o aplicativo Android não precisou ser modificada.

Melhorias no Cadastro e nas Configurações do Sistema

1. Objetivo

Esta etapa teve como objetivo melhorar a validação dos dados cadastrados no Django Admin, reduzir problemas durante o envio de imagens e organizar as configurações do backend para facilitar a execução do sistema em diferentes ambientes.

2. Validação do formulário de cadastro

Foi criado o arquivo **produtos/static/produtos/js/validacao_produto.js**, responsável por verificar os campos do formulário antes do envio ao Django.

A validação utiliza JavaScript para identificar campos obrigatórios não preenchidos e códigos de barras com formato inadequado, impedindo o envio nesses casos.

Essa implementação evita recarregamentos desnecessários da página, preservando a imagem selecionada quando são identificados erros simples de preenchimento.

As validações do backend foram mantidas. Erros identificados exclusivamente no servidor ainda podem provocar o recarregamento do formulário.

3. Validação flexível dos códigos de barras

Foi criado o módulo **produtos/validacao_codigos.py**, responsável por verificar o dígito verificador de códigos **EAN-8, EAN-13 e UPC-A**, utilizando o algoritmo módulo 10.

A validação foi integrada ao JavaScript do formulário e ao arquivo **produtos/admin.py**.

Quando o dígito verificador apresenta inconsistência, o sistema informa ao administrador sobre um possível erro de digitação, sem impedir o cadastro.

Essa abordagem permite a utilização de códigos internos dos estabelecimentos, mantendo a obrigatoriedade de códigos numéricos de 1 a 20 dígitos e a restrição de duplicidade.

Foram implementados oito testes automatizados específicos para o novo validador.

4. Organização das configurações do backend

O arquivo **core/settings.py** foi atualizado para permitir configurar o modo de execução do Django por meio da variável **DJANGO_DEBUG**, definida no **.env**.

Também foram revisadas as configurações existentes de conexão MySQL, hosts permitidos e armazenamento de imagens.

O arquivo **.gitignore** foi ajustado para impedir o versionamento de credenciais, ambientes virtuais, arquivos temporários e imagens armazenadas no diretório **media/**.

Foi preparado o arquivo **.env.example**, contendo a estrutura das configurações necessárias para executar o backend em outros ambientes, sem incluir credenciais reais.

A separação entre o endereço público das imagens **(IMAGENS_BASE_URL)** e o diretório físico de armazenamento **(IMAGENS_STORAGE_ROOT)** foi mantida.

5. Testes e validação

Foram realizados testes manuais no Django Admin, verificando o preenchimento obrigatório, a preservação da imagem selecionada, os avisos de dígito verificador e o cadastro de códigos internos.

Os testes automatizados do backend foram executados e aprovados após as alterações.

Também foram verificados os arquivos ignorados pelo Git e as configurações efetivamente carregadas pelo Django, incluindo DEBUG, ALLOWED_HOSTS, conexão MySQL e diretórios de imagens.

As verificações confirmaram o funcionamento das configurações no ambiente local de desenvolvimento.

6. Considerações finais

As melhorias aumentaram a confiabilidade do cadastro de produtos, reduziram erros de preenchimento e facilitaram a configuração do backend para diferentes instalações.

As alterações foram concentradas no Django Admin e nas configurações do servidor, sem necessidade de modificações na API REST ou no aplicativo Android.

Atualização e Revisão do Backend Django

1. Atualização das Dependências

Foi realizada a atualização do Django da versão **6.0.1 para 6.0.10**, incorporando correções de segurança e manutenção.

A versão foi atualizada no arquivo **requirements.txt**, mantendo as demais dependências do projeto.

Após a atualização, foram realizados testes de compatibilidade com o Django REST Framework e MySQL, além da execução dos testes automatizados existentes.

Todos os testes foram concluídos com sucesso, sem necessidade de alterações nos modelos ou na estrutura do banco de dados.

Aprimoramento da Validação e do Gerenciamento de Imagens

1. Objetivo

Esta etapa teve como objetivo melhorar a validação, o armazenamento e a exclusão das imagens dos produtos, garantindo maior consistência entre o banco de dados MySQL e os arquivos armazenados pelo backend.

2. Validação e Preparação das Imagens

Os arquivos **produtos/forms.py** e **produtos/imagens.py** foram aprimorados para realizar a validação e conversão integral das imagens antes do salvamento do produto.

Foi utilizado **BytesIO** para processar as imagens em memória, produzindo os dados finais no formato **WebP**.

Foram mantidas as seguintes regras:

- Formatos aceitos: JPEG, PNG e WebP;
- Tamanho máximo: 5 MB;
- Limite de 20 milhões de pixels na imagem original;
- Resolução máxima de saída: 1200 × 1200 pixels;
- Preservação da proporção, orientação EXIF e transparência.

Caso a imagem seja inválida ou apresente falha na conversão, o Django Admin informa o erro e impede o salvamento.

A implementação também evita conversões duplicadas da mesma imagem.

3. Publicação das Imagens

Foi implementada a função **publicar_imagem_produto()**, no arquivo **produtos/imagens.py**, separando o processamento da gravação definitiva.

O arquivo WebP é publicado utilizando um arquivo temporário e a função **os.replace()**, reduzindo o risco de gravações incompletas e preservando a imagem anterior caso ocorra falha antes da substituição.

4. Controle das Transações

O arquivo **produtos/admin.py** foi atualizado para utilizar **transaction.on_commit()**.

Com essa implementação, a publicação e a substituição das imagens ocorrem somente após a confirmação da transação do produto no MySQL.

Em caso de rollback, as imagens existentes permanecem preservadas.

Quando o código de barras é alterado, a imagem é publicada com o novo código antes da remoção do arquivo antigo.

Falhas posteriores ao commit são registradas para diagnóstico e eventual correção manual.

5. Proteção contra Imagens Órfãs

Foi adicionada uma validação ao formulário administrativo para identificar imagens armazenadas sem um produto correspondente.

Quando um cadastro ou alteração de código encontra um arquivo órfão, o sistema bloqueia a operação e apresenta uma mensagem ao administrador.

Essa verificação evita que imagens antigas sejam associadas incorretamente a novos produtos.

6. Exclusão Segura das Imagens

O método **delete_model()** foi aprimorado para verificar a associação do produto e a identificação do arquivo antes de removê-lo.

A exclusão da imagem continua ocorrendo somente após a confirmação da transação no MySQL.

Também foram adicionados registros de erros e avisos ao administrador quando a remoção não pode ser concluída.

A exclusão de produtos em lote permanece desabilitada.

7. Testes e Validação

Foram implementados novos testes automatizados para verificar:

- Validação e conversão WebP;
- Cadastro e edição de produtos;
- Substituição e exclusão de imagens;
- Alteração do código de barras;
- Operações de commit e rollback;
- Falhas de gravação e remoção;
- Proteção contra imagens órfãs.

A suíte de testes evoluiu de **23 para 115 testes automatizados**, todos aprovados.

Os testes foram executados utilizando banco MySQL e diretórios de imagens isolados, preservando os dados originais do sistema.

Também foram realizados testes manuais no Django Admin e de consulta pelo celular, confirmando o funcionamento das funcionalidades implementadas.

8. Considerações Finais

As melhorias aumentaram a confiabilidade das operações de cadastro, atualização e exclusão de produtos e imagens.

A arquitetura original foi preservada, sem necessidade de novas tabelas, serviços externos ou alterações no aplicativo Android.

Atualização da Documentação OpenAPI

1. Objetivo

Foi realizada a atualização da documentação da API REST para refletir corretamente os dados e as respostas disponibilizadas pelo backend.

2. Implementação

O arquivo **schema.yml** foi atualizado utilizando a biblioteca **drf-spectacular**.

No arquivo **produtos/views.py**, foi ajustada a anotação com **extend_schema_view()**, permitindo documentar corretamente o método HTTP GET.

O schema passou a representar:

- **HTTP 200:** produto encontrado;
- **HTTP 400:** código de barras inválido;
- **HTTP 404:** produto não encontrado;
- Campo imagem: URL ou null;
- Campo preco: texto decimal;
- Demais informações do produto e estrutura das respostas de erro.

O contrato da API REST permaneceu inalterado.

3. Testes e Validação

A especificação OpenAPI 3.0.3 foi gerada e validada com o comando:

python manage.py spectacular --file schema.yml --validate --fail-on-warn

A validação foi concluída sem avisos.

Também foram verificadas as respostas da API para produtos com imagem, sem imagem, códigos inválidos e produtos inexistentes.

Os 115 testes automatizados foram executados novamente, todos aprovados.

4. Considerações Finais

A documentação OpenAPI foi atualizada e validada, mantendo a compatibilidade com o aplicativo Android.

Revisão Final do Backend

1. Objetivo

Foi realizada uma revisão técnica do backend para verificar a organização do código, o funcionamento das implementações e a confiabilidade das operações.

2. Componentes Revisados

Foram analisados:

- Configurações do Django;
- Models e migrations;
- Django Admin e formulários;
- Validação dos códigos de barras;
- Processamento e armazenamento de imagens;
- Transações MySQL;
- API REST, views e serializers;
- Documentação OpenAPI;
- Dependências e testes automatizados.

A revisão confirmou a separação adequada das responsabilidades entre os componentes e o funcionamento das melhorias implementadas.

Não foram identificados problemas críticos ou de alta prioridade que impedissem a continuidade do projeto.

3. Testes Automatizados

A suíte final foi executada utilizando o comando:

python manage.py test produtos --verbosity 2

Resultados:

- Testes executados: **115**;
- Testes aprovados: **115**;
- Falhas: **0**.

Também foram realizadas verificações de configuração, compatibilidade das dependências, integridade das migrations e validação da documentação OpenAPI.

Os dados e as imagens originais foram preservados durante os testes.

4. Considerações Finais

As funcionalidades de consulta de produtos, administração dos registros, gerenciamento de imagens e comunicação por API REST permanecem implementadas e testadas.

Testes Integrados Finais do Sistema

1. Objetivo

Após a conclusão das revisões do backend e do aplicativo Android, foram realizados os testes integrados finais do sistema Verificador de Preços.

O objetivo foi verificar o funcionamento conjunto do aplicativo Android, da API REST desenvolvida em Django e do banco de dados MySQL.

Foram avaliadas as consultas de produtos, a leitura de códigos de barras, a exibição de imagens, a atualização de informações e o tratamento de erros.

2. Ambiente de Testes

Os testes foram realizados utilizando:

- **Backend:** Python, Django 6.0.10 e Django REST Framework;
- **Banco de dados:** MySQL 8.0.46;
- **Aplicativo Android:** Kotlin, Jetpack Compose e arquitetura MVVM;
- **Comunicação:** Retrofit e Moshi;
- **Leitura de códigos:** CameraX e ML Kit;
- **Exibição de imagens:** Coil;
- **Dispositivo:** Motorola Moto G22, Android 12;
- **Rede:** Wi-Fi local.

O servidor Django foi executado através do comando:

python manage.py runserver 0.0.0.0:8000 --noreload

O aplicativo acessou a API através do endereço:

http://192.168.100.10:8000/

3. Procedimentos e Resultados

Para os testes de cadastro, atualização e exclusão, foram utilizados produtos temporários, preservando os registros originais do sistema.

Foram realizados **13 testes integrados**:

Nº	Teste realizado	Resultado
1	Comunicação pela rede Wi-Fi	Aprovado
2	Consulta manual de produtos	Aprovado
3	Leitura de código de barras pela câmera	Aprovado
4	Exibição de produtos com e sem imagem	Aprovado
5	Atualização do preço pelo Django Admin	Aprovado
6	Substituição da imagem do produto	Aprovado
7	Alteração do código de barras	Aprovado

8	Consulta de produto inexistente	Aprovado
9	Tratamento de código de barras inválido	Aprovado
10	Função Nova leitura	Aprovado
11	Tratamento de servidor indisponível	Aprovado
12	Exclusão individual de produto	Aprovado
13	Remoção dos registros temporários de teste	Aprovado

Durante os testes, foi confirmado que as alterações realizadas no Django Admin são refletidas corretamente nas consultas do aplicativo Android.

A substituição das imagens também funcionou conforme esperado, exibindo os arquivos atualizados após uma nova consulta.

Os tratamentos de erros apresentaram as mensagens correspondentes, e a câmera continuou funcionando após sucessivas operações de leitura.

Ao final, os produtos temporários utilizados nos testes foram removidos.

4. Testes Automatizados

Antes da execução dos testes integrados, foram realizadas as verificações automatizadas dos dois projetos.

Backend Django:

- 115 testes executados;
- 115 testes aprovados;
- Nenhuma falha.

Aplicativo Android:

- 25 testes unitários executados;
- 25 testes aprovados;
- Compilação concluída com sucesso;
- Android Lint sem erros.

A documentação OpenAPI 3.0.3 também foi validada com sucesso.

Somando as suítes do backend e do Android, foram aprovados **140 testes automatizados**, além dos 13 testes integrados realizados no dispositivo físico.

5. Considerações Finais

Todos os 13 testes integrados apresentaram os resultados esperados, sem falhas relatadas.

Foi confirmado o funcionamento completo do fluxo principal do sistema:

Leitura do código de barras → Aplicativo Android → API REST Django → Banco de dados MySQL → Retorno e exibição das informações do produto.

Os resultados demonstraram que o sistema consegue consultar produtos, apresentar preços e imagens atualizadas e tratar situações de erro de maneira adequada.

Com isso, a implementação e a validação funcional do Verificador de Preços foram concluídas com sucesso no ambiente de testes utilizado.

Descoberta Automática do Servidor Django por mDNS

1. Objetivo

Após os testes integrados, foi implementada a descoberta automática do servidor na rede local. O objetivo foi permitir que o aplicativo Android encontre o endereço do Django sem exigir a configuração manual do IP a cada mudança de rede.

2. Implementação no Backend

O arquivo **anunciar_servidor.py** utiliza a biblioteca zeroconf para publicar o serviço Verificador de Precos na rede por mDNS/DNS-SD. O anúncio informa o IPv4 do computador, a porta 8000, o tipo _vprecos._tcp.local. e os metadados app=verificador-precos e version=1.0.

Antes de anunciar o serviço, o script valida o IPv4 informado e verifica se o Django está acessível pela rota /api/schema/. O anúncio é executado separadamente do Django e é encerrado com Ctrl+C, liberando os recursos de rede.

Também foram criados verificar_descoberta.py, para localizar os anúncios disponíveis, e **test_descoberta_servidor.py**, para a validação automatizada.

3. Testes

Foram aprovados 33 testes automatizados da descoberta, além da verificação real de anúncio, localização e desregistro no computador. Posteriormente, o anúncio também foi encontrado pelo Motorola Moto G22 na mesma rede Wi-Fi.

Descoberta do Servidor no Aplicativo Android

1. Implementação

O aplicativo Android passou a utilizar NsdManager para procurar o serviço anunciado pelo backend. A implementação foi organizada em componentes próprios para descoberta, estados de execução e interface, sem misturar essa responsabilidade com a consulta de produtos.

A descoberta verifica o tipo do serviço, os metadados TXT, o IPv4 e a porta. A tela informa quando está procurando, quando encontra um servidor e quando há falhas. O usuário pode interromper ou repetir a busca.

No Android 12, é utilizado MulticastLock durante a descoberta. O recurso é liberado ao encerrar a busca. O aplicativo também possui tratamento previsto para a permissão ACCESS_LOCAL_NETWORK em versões Android que a exigem.

2. Validação

No Moto G22, a descoberta foi testada com o anúncio ligado e desligado. Foram confirmados o endereço encontrado, a repetição da busca, o encerramento sem travamentos e a retomada do aplicativo após ir para segundo plano.

Conexão Dinâmica do Android com a API REST

1. Configuração Automática

Após descobrir o Django, o aplicativo verifica a compatibilidade da API por HTTP e consulta o schema OpenAPI. Quando o servidor é validado, seu endereço passa a configurar a conexão Retrofit, substituindo a dependência de um IP fixo definido durante a compilação.

O sistema controla a sessão ativa para evitar que respostas antigas sejam exibidas após a troca de servidor. Quando vários anúncios compatíveis são encontrados, a seleção é apresentada ao usuário.

2. Configuração Manual

Foi mantida uma alternativa para redes que bloqueiam mDNS. O usuário pode informar manualmente um endereço e uma porta; antes de salvar a configuração, o aplicativo verifica se a API está acessível e é compatível. Ao abrir novamente o aplicativo, o servidor manual salvo é revalidado.

3. Preservação das Consultas

A leitura por câmera com CameraX e ML Kit, a digitação manual do código de barras e a exibição de informações dos produtos foram preservadas. A comunicação utiliza Retrofit e Moshi, seguindo a arquitetura MVVM existente.

Endereços Independentes para API e Imagens

1. Separação das Responsabilidades

O mDNS informa exclusivamente o endereço do servidor Django, responsável pelas consultas REST. O endereço público das fotografias é definido separadamente no backend pela variável IMAGENS_BASE_URL do arquivo **.env**.

Na resposta da API, o campo imagem contém a URL completa gerada pelo Django ou o valor null, quando o produto não possui fotografia. O Android entrega essa URL diretamente ao Coil, sem substituir o IP ou a porta pelo endereço da API.

2. Compatibilidade

Assim, o Django e as imagens podem ser disponibilizados pelo mesmo servidor ou por servidores diferentes. O endereço configurado precisa estar acessível ao celular, e o servidor de mídia deve disponibilizar os arquivos WebP nos caminhos informados pelo backend.

A versão presente no parâmetro **“?v=...”** que é responsável por controlar a versão da imagem de um produto e evitar problemas com o cache. é preservada para permitir a atualização das imagens. Se uma fotografia estiver indisponível, os dados de nome, descrição e preço continuam visíveis no aplicativo.

3. Testes

Foram executados testes automatizados com URLs de imagens no mesmo endereço da API e em endereços diferentes, além de imagens ausentes. No Moto G22, foram confirmadas consultas e exibição de imagens WebP após a integração dinâmica.

Inicialização Simplificada do Backend no Windows

1. Objetivo

Para facilitar a execução do sistema, o arquivo iniciar-servidor.bat passou a ser o ponto único de entrada do backend no Windows. Ele utiliza o Python do ambiente virtual e chama iniciar_servidor.py, responsável pelo menu de inicialização.

2. Modos Disponíveis

LOCAL (OFF-LINE): inicia somente o Django no endereço 127.0.0.1:8000, sem publicar o anúncio mDNS.

REDE (ON-LINE): lista as interfaces de rede e seus endereços IPv4, permite selecionar o IP ou digitá-lo manualmente e abre dois terminais separados, identificados como Django e mDNS. Neste modo, o anúncio aguarda a API ficar disponível antes de iniciar.

SAIR: encerra o menu sem iniciar os serviços. Nos dois modos, o banco MySQL precisa estar disponível. O modo rede utiliza a rede local e não exige acesso à internet.

3. Testes

Foram aprovados 24 testes automatizados do inicializador. A execução real do arquivo .bat também foi validada no Windows, incluindo seleção da opção de rede, abertura dos dois terminais, anúncio ativo, descoberta pelo Moto G22 e encerramento com Ctrl+C.

Personalização Visual do Aplicativo

1. Ícone do Aplicativo

O ícone padrão do Android foi substituído por uma arte personalizada relacionada à leitura de códigos de barras e à verificação de preços. Foram configurados recursos adaptativos, ícones tradicionais e uma representação monocromática para ícones temáticos.

2. Assinatura ATL Software

Foi criado o componente reutilizável **AssinaturaATL.kt**, que exibe o símbolo </> com degradê verde e o texto “Desenvolvido por ATL Software”. O símbolo utiliza Canvas, Path e Brush.linearGradient, com fundo transparente.

A assinatura aparece no final do conteúdo rolável das telas de leitura, resultado do produto e conexão com o servidor. O componente utiliza símbolo de 32 × 22 dp e texto de 12 sp, sem sobrepor os botões ou a câmera.

3. Verificação Visual

A assinatura foi validada no Moto G22 nas três telas. Foram confirmados o degradê, a legibilidade, o alinhamento, a rolagem e a manutenção das consultas de produtos. A implementação não alterou as regras de negócio ou os serviços do backend.

Testes Complementares e Estado Atual do Sistema

1. Testes Automatizados

Após as melhorias, os relatórios de execução registraram 115 testes aprovados do aplicativo produtos no Django, 33 testes do serviço de descoberta e 24 testes do inicializador Windows.

No Android, a suíte passou a contar com 141 testes unitários aprovados, sem falhas relatadas. A compilação :app:assembleDebug foi concluída com sucesso, e o Android Lint apresentou zero erros, mantendo 35 avisos existentes.

2. Testes no Dispositivo Físico

No Motorola Moto G22 com Android 12, foram confirmados a descoberta automática, a conexão com a API, a consulta por câmera e por digitação, a exibição de preços e imagens, a configuração manual e a recuperação de situações de indisponibilidade. A inicialização simplificada do servidor e a assinatura ATL também foram validadas em testes manuais.

3. Limitações e Próximas Etapas

O protótipo foi validado na rede Wi-Fi local. A descoberta mDNS depende do suporte a multicast e das configurações da rede. Ainda não foram realizados testes físicos com API e imagens hospedadas em servidores separados nem em um dispositivo com Android 17.

A versão debug foi compilada e testada no celular. A preparação de um APK de release assinado e do manual de instalação do avaliador permanece como etapa de entrega, não sendo apresentada aqui como concluída.

4. Considerações Finais

Com a descoberta automática, a conexão dinâmica, a inicialização simplificada e a personalização visual, o Verificador de Preços passou a oferecer um fluxo mais prático de instalação e uso. Os testes realizados confirmaram o funcionamento das funcionalidades principais no ambiente de desenvolvimento e no dispositivo de validação.